Ders 3 Genişletilmiş Notu: Sonlu Otomatlara Giriş, DFA'nın Biçimsel Tanımı, Dil Bulma ve DFA Tasarımı

Ders 3 Genişletilmiş Notu: Sonlu Otomatlara Giriş, DFA'nın Biçimsel Tanımı, Dil Bulma ve DFA Tasarımı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sonlu Otomat Nedir?
- 2. Bir Makineyi Bir Dizgi Üzerinde Çalıştırmak
- 3. Sonlu Otomatın Biçimsel Tanımı
- 4. Tasarım Örneği: Tek Sayıda 1 İçeren Dizgiler
- 5. Deterministik Sonlu Otomat (DFA)
- 6. Kabul ve Ret Koşulları
- 7. Verilen Bir Makinenin Dilini Bulmak
- 8. Genişletilmiş Geçiş Fonksiyonu
- 9. Durum Sayısından Uzun Dizgiler ve Döngü
- 10. Bir DFA'nın Kabul Ettiği ve Reddettiği Dil
- 11. Özel Diller: Boş Küme, Bütün Dizgiler, Yalnızca Boş Dizgi
- 12. Tasarım Örneği: `ab` Önekli Dizgiler
- 13. Tasarım Örneği: `001` İçeren Dizgiler
- 14. Tasarım Örneği: `001` İçermeyen Dizgiler
- 15. Evde Çalışma Örneği: awa Dili
- 16. Düzenli Diller ve Düzenli Olmayan Diller

1. Sonlu Otomat Nedir?

Bu hafta sonlu otomatlara (finite automata) başlıyoruz. Sonlu otomatlar mantıksal makinelerdir. Bir sonlu otomatın giriş şeridi (input tape) vardır ve bu şeritte bir dizgi (string) yazılıdır. Makine bu dizginin sembollerini birer birer okur. Bütün semboller okunduktan sonra makine iki çıktıdan birini verir: kabul (accept) ya da ret (reject). Başka bir çıktı yoktur.

1.1 Durumlar ve Geçişler

Sonlu otomatın içinde durumlar (states) ve durumların arasında geçişler (transitions) vardır. Bu derste durumlara genellikle $q_0, q_1, q_2, \ldots$ adını vereceğiz. Bu zorunlu değildir, durumlara başka adlar da verebilirsiniz.

Her geçiş, makine bir sembol okuduğunda gerçekleşir. Örneğin q_0 durumundaki makine `a` okuyunca q_1 durumuna geçebilir. Her durum için alfabedeki her sembolde ne olacağını bilmemiz gerekir. Hocanın ilk örneğinde alfabe yalnız `a` ve `b`'den oluşuyor, dolayısıyla her durum için iki olasılık vardır.

Durum diyagramında üç işaret önemlidir:

- **Başlangıç durumu (initial state):** Herhangi bir durumdan çıkmayan, kaynağı olmayan bir okla gösterilir. Makine hesaplamaya bu durumdan başlar.

- **Kabul durumu (accepting state):** Çift çemberle gösterilir. Makine bu durumda biterse giriş şeridindeki dizgiyi kabul eder.
- **Ret durumu (reject state):** Tek çemberle gösterilir.

  Ders sonrası not: Hoca, bir makinede bir ya da daha çok kabul durumu ve bir ya da daha çok ret durumu olabileceğini söyledi. Doğrusu, ikisinin sayısı sıfır da olabilir. Bölüm 11'deki makinelerin birinde hiç kabul durumu, diğerinde hiç ret durumu yoktur.

1.2 İlk Örnek Makine

Alfabe $\Sigma = \{a, b\}$, durumlar $q_0, \ldots, q_5$, başlangıç durumu q_0 ve tek kabul durumu q_4 olsun. Geçişler şöyledir:

- q_0: `a` ile q_1'e, `b` ile q_5'e gider.
- q_1: `b` ile q_2'ye, `a` ile q_5'e gider.
- q_2: `b` ile q_3'e, `a` ile q_5'e gider.
- q_3: `a` ile q_4'e, `b` ile q_5'e gider.
- q_4: `a` da `b` de okusa q_5'e gider.
- q_5: `a` da `b` de okusa q_5'te kalır.

Hoca q_5'in kendine dönen geçişini çizmeyi önce unuttu, makineyi çalıştırırken fark edip ekledi. Bu eksik önemlidir, çünkü her durum için her sembolde ne olacağını göstermek zorundayız.

2. Bir Makineyi Bir Dizgi Üzerinde Çalıştırmak

2.1 Okuma Kafası

Makinenin çalışmasını izlemek için bir okuma-yazma kafası (read-write head) düşünün. Kafa başlangıçta şeridin ilk sembolünü gösterir. Gösterdiği sembol, bir sonra okunacak semboldür. Sembolü okuduktan sonra bir sağa kayar.

> **Ders sonrası not:** Hoca kafaya "read-write head" dedi. Sonlu otomat şeride hiçbir şey yazmaz, yalnızca okur. Geçiş fonksiyonunun çıktısı da yalnızca yeni durumdur. Bu yüzden burada kafa fiilen yalnızca okuma kafasıdır.

2.2 `abbaaabb` Dizgisi Adım Adım

Şeritte `abbaaabb` yazılı olsun. Her adımı bir zaman adımı (time step) olarak düşünelim. Sıfırıncı anda durum q_0'dır, çünkü başlangıç durumu budur. Kafa da şeridin başındadır.

Adım	Önceki durum	Okunan sembol	Yeni durum
1	q_0	`a`	q_1
2	q_1	`b`	q_2
3	q_2	`b`	q_3
4	q_3	`a`	q_4
5	q_4	`a`	q_5
6	q_5	`a`	q_5
7	q_5	`b`	q_5
8	q_5	`b`	q_5

Dördüncü adımın sonunda q_4 durumundayız ve bu bir kabul durumudur. Fakat burada "makine kabul etti" diyemeyiz, çünkü şeritte henüz işlenmemiş semboller vardır.

> **Hocanın vurgusu:** Sonlu durum makineleri giriş şeridindeki bütün semboller okunana kadar devam eder. Bir kabul durumuna ulaştık diye durmayız.

Son sembol okunduğunda giriş tükenir ve hesaplama biter. Son durum q_5'tir. q_5 tek çemberlidir, yani ret durumudur. Dolayısıyla makine `abbaaabb` dizgisini reddeder.

> **Ders sonrası not:** Hoca bu makinenin dilini ayrıca söylemedi. Tablo, 10 sembole kadar bütün dizgilerde Python ile tarandı. Kabul edilen tek dizgi `abba`'dır. Yani $L = \{abba\}$.

2.3 Makine Aynı Anda Birden Çok Sembol Okuyabilir mi?

Soru: Bilgisayarda RAM'den veri okunurken veri yolunun (bus) genişliği birden fazla olabiliyor. Sonlu makine de bir adımda `ab` ya da `ba` gibi birden çok sembol okuyabilir mi, yoksa yalnızca tek sembol mü okumak zorunda?

Cevap: Makine her adımda yalnızca tek sembol okur. Program mantığınızda iki sembol okumayı göstermeniz gerekiyorsa iki yol vardır:

- q_0'da `ab` gelince q_2'ye gidilir diye kısaca yazabilirsiniz. `ab` okumak, önce `a` sonra `b` okumaya denktir. Bunun örneklerini göreceğiz.
- `ab` tek bir sembol gibi davranmalıysa, alfabeye yeni bir sembol eklersiniz. Örneğin `z` sembolü `ab` anlamına gelir.

Sonuçta sonlu durum makineleri bir seferde tek sembol okur. Hoca, öğrencinin "birden çok durum okuyor gibi gösterebiliriz" sözünü de düzeltti: Okunan şey durum değil, semboldür.

3. Sonlu Otomatın Biçimsel Tanımı

Durum diyagramı biz insanlar için çok yararlıdır. Fakat bunları gerçek bilgisayarlara verirken bir gösterime ihtiyaç duyarız. Bunun için biçimsel tanım (formal definition) kullanılır.

3.1 5'li Tanım

Bir sonlu otomat bir 5'lidir (5-tuple):

$$M = (Q, \Sigma, \delta, q_0, F)$$

Burada M otomatın adıdır. Hoca, demet (tuple) sözcüğünün basitçe bir liste demek olduğunu söyledi. 5'li, beş şeyden oluşan bir listedir. Programlama dillerindeki farkı da hatırlattı: Demetin içeriği değişmez, yani demet değiştirilemez (immutable) bir listedir.

> **Ders sonrası not:** Hoca bu demet kavramına Python ve JavaScript'i örnek gösterdi. Python için bu doğrudur. JavaScript'in standart dilinde ayrı bir demet türü büyük olasılıkla yoktur, bu yüzden örnek olarak Python'u esas alın.

Bileşenler ilk örnek makine üzerinden şöyledir:

Bileşen	Anlamı	İlk örnekteki değeri
Q	Durumlar kümesi	$\{q_0, q_1, q_2, q_3, q_4, q_5\}$
Σ	Alfabe	$\{a, b\}$
δ	Geçiş fonksiyonu (transition function)	Aşağıdaki tablo
q_0	Başlangıç durumu	q_0
F	Kabul durumları kümesi	$\{q_4\}$

Giriş şeridinde yalnızca alfabedeki semboller bulunabilir, başka sembol olamaz.

Başlangıç durumu için iki noktaya dikkat edin. Birincisi, başlangıç durumu bir küme değildir. Yalnızca bir tane olabilir, bu yüzden küme gösterimi yerine doğrudan durumun adı yazılır. Hoca önce küme parantezi kullandı, sonra bunu düzeltti. İkincisi, 5'lideki "q_0" demetin bileşeninin adıdır, kümenin içindeki q_0 ise bir durumdur. İkisi farklı şeylerdir. Durumlara başka adlar da verebilirdiniz. Örneğin başlangıç durumunun adı b_{00} olsaydı, 5'lideki başlangıç durumu bileşeninin değeri b_{00} olurdu.

F ise bir kümedir, çünkü birden çok kabul durumu olabilir. Bu örnekte yalnızca q_4 vardır.

3.2 Bir Soru: Geçişler İstenen Yere Gidebilir mi?

Soru: q_5'ten q_0'a bir geçiş konabilir mi?

Cevap: Evet. Gerekiyorsa q_5'ten q_0'a geçiş koyabilirsiniz. Uygun gördüğünüz her geçişi koyabilirsiniz.

Soru: Lojik dersinde gördüğümüz makineler de bu makineler mi oluyor?

Cevap: Hoca bu konuda bir şey söyleyemeyeceğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Altyazıda "melimur" diye geçen ifade büyük olasılıkla Mealy ve Moore makineleridir. Bunlar da sonlu sayıda durumu olan makinelerdir. Farkları, her adımda ya da her durumda bir çıktı üretmeleridir. Bu derste tanımlanan sonlu otomatın çıktısı ise yalnızca dizginin sonundaki kabul ya da rettir.

3.3 Geçiş Fonksiyonu ve Geçiş Tablosu

Diyagramı bir çizge gibi düşünürsek Q düğümleri, q_0 başlangıç düğümünü, F kabul düğümlerini verir. Kenarları, yani geçişleri ise δ verir:

$$\delta : Q \times \Sigma \to Q$$

Bu gösterim δ'nın bir fonksiyon olduğunu söyler. Tanım kümesi (domain) Q ile Σ'nın kartezyen çarpımıdır (cross product). Yani fonksiyon bir durum ve bir sembol alır, bir durum verir.

> **Ders sonrası not:** Hoca Q için "range" sözcüğünü kullandı. Fonksiyon gösteriminde Q değer kümesidir (codomain). Görüntü kümesi (range) daha küçük olabilir. İlk örnek makinede hiçbir geçiş q_0'a gelmez, görüntü kümesi $\{q_1, \dots, q_5\}$'tir.

Fonksiyon sadece bir eşlemedir (mapping), bu yüzden farklı biçimlerde gösterilebilir. Burada geçiş tablosu (transition table) kullanalım. Satırlar durumları, sütunlar sembolleri gösterir. Bir hücre, o durumdaki makine o sembolü okuyunca gidilen durumdur.

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

Bu tablodaki bilgi diyagramdaki bilgiye denktir. Diyagram görsel bir gösterimdir, ama makineleri ve bilgisayarları programlarken görsel gösterim kullanamayız. Bu yüzden 5'liyi kullanacağız.

> **Hocanın vurgusu:** Bir sonlu otomatın 5'li tanımı sorulduğunda bunu yazabilmelisiniz.

3.4 Geçiş Fonksiyonu Makinenin Programıdır

Sonlu otomatın bir makine olduğunu söylüyoruz. "Makine bunu okuyunca, şunu okuyunca" diye konuşuyoruz. Bir makinenin programı vardır. Burada program geçiş fonksiyonudur. Geçiş fonksiyonunu değiştirirseniz, yani diyagramdaki geçişleri değiştirirseniz, makinenin programını değiştirmiş olursunuz. Bir bakıma diğer dört bileşen makineyi oluşturur, geçiş fonksiyonu da orada kullanılan programdır.

> **Hocanın vurgusu:** Bundan sonra, istenen işe uygun sonlu durum makinesi tasarlayabilmelisiniz.

4. Tasarım Örneği: Tek Sayıda 1 İçeren Dizgiler

Şu kümeyi ele alalım:

$$A = \{w \mid w \text{ tek sayıda 1 içeren ikili dizgidir}\}$$

Görev, A'nın üyelerini kabul eden bir sonlu otomat tasarlamaktır. Burada elimizde tek bir dizgi yok, bir dizgi kümesi var. Genelde yapacağımız iş budur: Bir kümenin bütün üyelerini kabul eden otomat tasarlamak.

4.1 Önce Kümeyi Anlamak

Üyeler ikili dizgilerdir, yani alfabe $\{0, 1\}$'dir. Hoca birkaç üye yazdı: `0001`, `1`, `1000`, `111`, `0101001`. Bunların ortak özelliği, 1'lerin sayısının tek olmasıdır. Bu küme sonsuzdur.

Hoca dizgilerin çok uzun olabileceğini de vurguladı ve milyonlarca sembollük dizgiler olabileceğini söyledi. Bunun için `01111000111000001111` benzeri uzun bir dizgi yazdı.

> **Ders sonrası not:** Hoca bazı üyelerde "sonsuz sayıda sembol" olabileceğini söyledi. Doğrusu, her dizgi sonludur ama uzunluğunun bir üst sınırı yoktur. Sonsuz olan, dizginin kendisi değil kümedir. Ayrıca altyazıda okunduğu haliyle bu uzun dizgide 10 tane 1 vardır. Dizgi yalnız uzunluğu göstermek için söylendi, A'nın üyesi değildir.

4.2 Döngü Gereği

Üyeler keyfî uzunlukta olabildiği için düz, döngüsüz bir durum dizisi tasarlayamayız. Döngü yoksa keyfî sayıda sembolü kabul edemeyiz. Demek ki makinede döngü olmalıdır.

4.3 Saymak Yerine Çift mi, Tek mi Olduğunu Tutmak

Başlangıç durumundan yola çıkalım ve soralım: 0 okursam ne olur, 1 okursam ne olur? Aslında istediğimiz 1'leri saymaktır. 0'lar bunu etkilemez, öyleyse 0 okununca hiçbir şey değişmesin.

Hocanın fikri şudur: Bir şey saymak yerine, 1'lerin sayısının çift mi tek mi olduğunu düşünelim. Bu iki bilgiyi birer durumla tutalım.

- q_{even} durumu, şimdiye kadar çift sayıda 1 okunduğunu gösterir. Başlangıç durumu budur. 0 okununca bu durumda kalınır.
- q_{odd} durumu, şimdiye kadar tek sayıda 1 okunduğunu gösterir. Kabul durumudur. 0 okunnca bu durumda kalınır.
- q_{even}'dayken 1 gelirse q_{odd}'a, q_{odd}'dayken 1 gelirse q_{even}'a gidilir. Çünkü yeni bir 1, çift ile tek arasındaki durumu değiştirir.

Fikir özetle şudur: 1'lerin sayısının tek mi çift mi olduğunu göstermek için durumları kullandık.

4.4 Biçimsel Tanım

Makinenin 5'lisi şöyledir:

$$M = (\{q_{even}, q_{odd}\}, \{0, 1\}, \delta, q_{even}, \{q_{odd}\})$$

δ	0	1
q_{even}	q_{even}	q_{odd}
q_{odd}	q_{odd}	q_{even}

Biçimsel tanımı yazmak biraz iş ister ama zor bir tarafı yoktur.

4.5 `01001` Dizgisi ve Bir Yanlış Anlama

Soru: `01001` dizgisi tek bir sayıdır. Makine bunu kabul etmiyor gibi görünüyor, neden?

Hoca soruyu cevaplamak için makineyi `01001` üzerinde çalıştırdı:

Okunan sembol	Yeni durum
0	q_{even}
1	q_{odd}
0	q_{odd}
0	q_{odd}
1	q_{even}

Bütün semboller okununca son durum q_{even}'dır. Bu bir ret durumudur, makine dizgiyi reddeder. Burada bir yanlışlık yoktur.

Cevap: Dil, sayının sayısal olarak tek olmasına bakmaz. Tanımda "tek sayıda 1 içeren ikili dizgi" yazıyor. `01001` ikili sayı olarak 9'dur, yani tek bir sayıdır. Fakat içinde iki tane 1 vardır, bu yüzden dilde değildir.

4.6 Geçiş Fonksiyonunu Geçişler Kümesi Olarak Yazmak

Molanın ardından hoca bu örneğe bir ekleme yaptı. Geçiş fonksiyonu tablo yerine geçişlerin kümesi olarak da yazılabilir:

$$\delta = \{(q_{even}, 0, q_{even}),\ (q_{even}, 1, q_{odd}),\ (q_{odd}, 0, q_{odd}),\ (q_{odd}, 1, q_{even})\}$$

Buradaki her üçlü tek bir geçiştir. Örneğin ilk üçlü şunu söyler: q_{even} durumundayken 0 gelirse durum q_{even} olur. Tablo ve geçişler kümesi aynı fonksiyonun iki farklı yazılışıdır.

5. Deterministik Sonlu Otomat (DFA)

Sonlu otomatların iki türü vardır: deterministik (deterministic) ve deterministik olmayan (nondeterministic). Deterministik sonlu otomatta (deterministic finite automaton, DFA) her durum için alfabedeki her sembolün bir geçişi vardır.

Hoca buna önceki örneğe benzeyen, ama bir geçişi eksik olan iki durumlu bir makine gösterdi ve sordu: Bu makine deterministik midir? Değildir, çünkü q_1'deyken 0 gelirse ne olacağı yazılmamıştır.

> **Ders sonrası not:** Hoca tanımı "her durumda her sembol için bir geçiş" diye söyledi. Tam ifade şudur: Her durumda her sembol için **tam olarak bir** geçiş vardır. δ'nın $Q \times \Sigma \to Q$ biçiminde bir fonksiyon olması da bunu söyler.
>
> **Hocanın vurgusu:** "DFA tasarlayın" dendiğinde buna dikkat edin. Hiçbir geçişi atlayamazsınız.

Hoca iki temel beceriyi de ayırdı. Birincisi bir sistemi, yani makineyi tasarlamaktır. İkincisi verilen bir makinenin kabul ettiği dili bulmaktır.

6. Kabul ve Ret Koşulları

Makine ne zaman kabul eder, ne zaman reddeder?

- **Kabul:** Makine bütün girişi taramalı ve son durum kabul durumu olmalıdır.
- **Ret:** Makine bütün girişi taramalı ve son durum ret durumu, yani kabul olmayan bir durum olmalıdır.

Bir durum kabul durumu değilse ret durumudur. Yalnızca bu iki durum vardır.

7. Verilen Bir Makinenin Dilini Bulmak

Size makineler verilecek ve kabul ettikleri dil sorulacak. Hoca bunun için bir yöntem önerdi: Önce örnek dizgiler yazın, ne olduğunu anlamaya çalışın, deseni kavrayın, sonra deseni yazın.

> **Hocanın vurgusu:** Bir makinenin kabul ettiği dili yazabilmelisiniz.

7.1 Birinci Örnek: Üç Dizgilik Sonlu Dil

Alfabe $\{a, b\}$, durumlar $q_0, \ldots, q_5$. Kabul durumları q_0, q_2 ve q_4'tür. Geçiş tablosu ilk makineyle aynıdır: `a`, `b`, `b`, `a` okunursa $q_0 \to q_1 \to q_2 \to q_3 \to q_4$ yolu izlenir, geri kalan her geçiş q_5'e gider ve q_5 kendine döner.

Dili adım adım bulalım:

- Başlangıç durumu kabul durumudur. Yani hiçbir sembol vermesek bile makine kabul eder. Bu, boş dizgiyi (null string) kabul ettiği anlamına gelir. Boş dizgi "hiç dizgi yok" demektir. Bunu ε ile göstereceğiz.
- `ab` verirsek q_2'ye gideriz. q_2 kabul durumudur, `ab` kabul edilir.
- `abba` verirsek q_4'e gideriz. q_4 kabul durumudur, `abba` kabul edilir.
- Başka hiçbir şey kabul edilmez. Örneğin yalnız `a` verirsek q_1'de kalırız ve q_1 ret durumudur. Hoca, `ab` ve `abba` dışında verilen her şeyin q_5'te bittiğini söyledi.

Sonuç:

$$L(M) = \{\varepsilon, ab, abba\}$$

> **Ders sonrası not:** Hocanın "geri kalan her şey q_5'te biter" sözünün iki istisnası vardır: `a` dizgisi q_1'de, `abb` dizgisi q_3'te biter. İkisi de ret durumu olduğu için sonuç değişmez. Tablo 10 sembole kadar bütün dizgilerde Python ile tarandı ve kabul edilenler tam olarak bu üç dizgi çıktı.

7.2 Tuzak Durum

Bu makinedeki q_5 gibi durumlara tuzak durum (trap state) ya da ölü durum (dead state) denir. Nedeni şudur: Bu durumda her sembol için kendine dönen bir döngü vardır. q_5'e bir kez gelirseniz, hangi sembol gelirse gelsin bu durumdan çıkamazsınız. Üstelik bu bir ret durumudur. Bu yüzden ölü durumdur.

7.3 İkinci Örnek: $a^n b$

Durumlar q_0, q_1, q_2, başlangıç q_0, tek kabul durumu q_1, alfabe $\{a, b\}$.

δ	a	b
q_0 (başlangıç)	q_0	q_1
q_1 (kabul)	q_2	q_2
q_2 (tuzak)	q_2	q_2

Bu makine boş dizgiyi kabul etmez, çünkü başlangıç durumu ret durumudur. q_0'da istediğimiz kadar `a` okuyabiliriz. Sonra `b` gelince kabul durumu olan q_1'e geçeriz. Yani `ab`, yalnız `b` ve `aaab` kabul edilir.

Buradaki ilginç nokta şudur: Kabul durumuna vardıktan sonra bir sembol daha gelirse makine ölü duruma geçer. Yani yalnızca bir tane `b` olabilir:

- `aabb` kabul edilmez. İlk `b` ile q_1'e, ikinci `b` ile q_2'ye gidilir.
- `aba` kabul edilmez. `ab`'ye kadar kabul durumundayız, ama `a` ile q_2'ye gidilir.

Bu dilin sonsuz sayıda üyesi vardır, dolayısıyla üyeleri tek tek listeleyemeyiz. Kısa bir biçimde yazmamız gerekir:

$$L = \{a^n b \mid n \geq 0\}$$

a^n, n tane a'nın art arda eklenmesi (concatenation) demektir. Ardından tek bir b gelir. Bu bir desendir: Bir dizgi desene uyuyorsa kümededir.

> **Hocanın vurgusu:** Çoğu zaman sonsuz kümelerle uğraşacağız. Sonlu kümeler sorun değildir, onlar düzenlidir (regular). Sonsuz kümeler için bu desen gösterimi çok kullanışlıdır.

7.4 Üçüncü Örnek: Çift Sayıda 1

İki durum var: q_0 ve q_1. Başlangıç durumu q_0 kabul durumudur. q_0'dan 1 ile q_1'e, q_1'den 1 ile q_0'a gidilir.

Hoca önce 0 geçişlerinin eksik olduğunu düşündü, çünkü alfabeyi ikili sanmıştı. Sonra düzeltti: Bu örnekte alfabe yalnız $\{1\}$'dir. Dolayısıyla makine bir DFA'dır, eksik geçiş yoktur.

Yine örnek dizgilerle başlayalım:

Dizgi	Son durum	Sonuç
ε	q_0	Kabul
1	q_1	Ret
11	q_0	Kabul
111	q_1	Ret
1111	q_0	Kabul

> **Ders sonrası not:** Altyazıda hoca bir yerde "one, one, one" dizgisinin kabul edildiğini söylüyor gibi görünüyor. Doğrusu, 111 reddedilir, 1111 kabul edilir.

Desen açıktır: 1'lerin sayısı çift olmalıdır. Hoca dili iki biçimde yazdı ve ikisinin de doğru olduğunu söyledi. Aynı şeyi söylemenin farklı yolları olabilir.

$$L = \{x \mid x \in \{1\}^* \text{ ve } |x| \text{ çifttir}\} \qquad L = \{1^n \mid n \text{ çifttir}\}$$

Hoca $n \geq 0$ yazmak gerekip gerekmediğini de tarttı. Sonra gerekmediğine karar verdi, çünkü 1'lerin sayısı negatif olamaz.

Soru: "n çift" dediğimizde $n = 0$ sorun olmaz mı?

Cevap: Olmaz. 1^0 boş dizgidir ve bu makine boş dizgiyi kabul eder. Sıfırı çift kabul ediyoruz.

> **Ders sonrası not:** Hoca sıfırın çift olmasını ortama bağlı bir varsayım gibi anlattı. Bu bir varsayım değildir. $0 = 2 \cdot 0$ olduğu için sıfır tanım gereği çifttir.

8. Genişletilmiş Geçiş Fonksiyonu

Geçiş fonksiyonunun genişletilmiş bir biçimi vardır: genişletilmiş geçiş fonksiyonu (extended transition function). Yıldızla gösterilir.

Normal δ bir durum ve alfabeden tek bir sembol alıp bir durum veriyordu. δ^* ise bir durum ve alfabeden üretilen bir dizgi alır, bir durum verir:

$$\delta : Q \times \Sigma \to Q \qquad \delta^* : Q \times \Sigma^* \to Q$$

$\delta^*(q, w) = q'$ yazılışı şunu söyler: q durumundayken w dizgisinin tamamı okunursa q' durumuna varılır.

Hoca bunun yalnızca kısa bir gösterim olduğunu, yeni bir şey söylemediğini belirtti. Örneğin q_0, q_1, q_3, q_4 gibi durumlardan oluşan bir yolda q_1'deyken önce b, sonra c okuyunca q_4'e varılıyorsa şöyle yazarız:

$$\delta^*(q_1, bc) = q_4$$

`bc` tek sembol değil, bir dizgidir. Bu gösterim aradaki durumu söylemez, çünkü ara durumu önemsemiyoruz.

Hoca bu örneği önce `cb` diye yazdı. Bir öğrencinin uyarısı üzerine `bc` olarak düzeltti.

> **Ek bilgi (derste anlatılmadı):** δ^* genellikle özyinelemeli olarak tanımlanır. $\delta^*(q, \varepsilon) = q$ ve $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$ olur. Yani boş dizgi durumu değiştirmez. Uzun bir dizgi de son sembolü dışındaki kısmı işlendikten sonra son sembolle bir adım daha ilerletilir.

9. Durum Sayısından Uzun Dizgiler ve Döngü

Hoca altı durumlu bir makine çizdi: $q_0, \ldots, q_5$. Bu makinede $\delta^*(q_1, bcb) = q_5$ olsun. Yani q_1'den `bcb` okununca q_5'e varılıyor. q_5 kabul durumu olsun ve ona gelindikten sonra `b`'ler bu durumda kalsın. Makine 12 sembollük bir dizgiyi kabul ediyor.

Durum sayısı 6, sembol sayısı 12'dir. Buradan şu sonuç çıkar:

> **Hocanın vurgusu:** Durum sayısından daha çok sembolü olan bir dizgiyi kabul etmek için makinede döngü olmalıdır. Bu ileriki derslerde çok önemli olacak. Bu yüzden şimdiden dikkat çekmek istiyorum.

Soru: q_0'dan q_1'e giderken neden hiçbir sembol yazılmadı? Olması gerekmiyor mu?

Cevap: Hızlı yazmak için atlandı. Kabul ettiğini söylemek için hesaplama q_0'dan başlamalıdır, oraya bir `c` koyalım.

Soru: Yukarıdaki δ^* örneğinde de `a` yazmamıştık. Orada da gerekiyor mu?

Cevap: Hayır. O örnek yalnızca "q_1'den başlayıp `bc` alırsa q_4'e gider" diyor. İstediğiniz yerden başlayabilirsiniz, q_0 yazmanız gerekmez. Burada ise makinenin dizgiyi kabul ettiğini anlatmak istiyoruz. Kabul için döngü gerekir, çünkü sembol sayısı durum sayısından fazladır. Döngü sayesinde daha büyük bir dizgi kabul edilebilir.

Soru: q_3'teyken `b` geldiğinde hem q_5'e hem q_4'e gidilebiliyor diye yazdık. Burada da bir yanlışlık yok mu?

Cevap: Var, haklısınız. DFA'da bir durum ve bir sembol için tek geçiş olur. Hoca örneği hızlıca uydurduğunu söyledi ve diyagramı düzeltti.

> **Ek bilgi (derste anlatılmadı):** Bu gözlem, güvercin yuvası ilkesinin bir uygulamasıdır. m sembollük bir dizgi okunurken $m+1$ durum ziyareti olur. Bu yüzden $m \geq k$ ise k durumlu makinede bir durum tekrar eder. Bu, 3 duruma kadar bütün iki sembollü DFA'larda taranarak da doğrulandı. Aynı fikir ileride pompalama önsavının (pumping lemma) temelidir.

10. Bir DFA'nın Kabul Ettiği ve Reddettiği Dil

10.1 Kabul Edilen Dil

Şimdiye kadar söylediğimizi biçimsel olarak yazalım. Bir M otomatının kabul ettiği dil $L(M)$ ile gösterilir ve M'nin kabul ettiği bütün dizgileri içerir. L bir fonksiyon gibidir: Ona bir makine verirsiniz, size bir küme verir.

Tersinden de söyleyebiliriz: $L(M) = L'$ ise L' dili M DFA'sı tarafından kabul edilir.

> **Hocanın vurgusu:** Bir otomat bir ve yalnız bir dil kabul eder. Fakat bir dil birden çok otomat tarafından kabul edilebilir.

$M = (Q, \Sigma, \delta, q_0, F)$ bir DFA olsun. Kabul ettiği dil şudur:

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$$

Burada Σ^* bu alfabeyle elde edilebilecek bütün dizgilerin kümesidir. Başlangıç durumundan başlayıp w verdiğimizde, son vardığımız q' durumu F'nin içindeyse w dildedir. Hoca formülü önce eşitlik gibi yazmaya başladı, sonra küme üyeliği olarak yazdı.

10.2 Reddedilen Dil

Σ^* burada bir evrensel küme (universal set) gibidir. Makinenin reddettiği dizgilerin kümesini aynı gösterimle kolayca yazabiliriz. Bu, $L(M)$'nin tümleyenidir (complement):

$$\overline{L(M)} = \{w \in \Sigma^* \mid \delta^*(q_0, w) \notin F\}$$

Evrensel kümenin içinde makinenin kabul ettiği dizgiler onun dilidir. Geri kalanlar dilin tümleyenidir.

11. Özel Diller: Boş Küme, Bütün Dizgiler, Yalnızca Boş Dizgi

İkinci moladan sonra hoca bu tanımları örneklerle pekiştirdi. Üç örnekte de alfabe $\{a, b\}$'dir.

Hiç kabul durumu olmayan makine. Tek durum q_0 var, `a` ve `b` ile kendine dönüyor, kabul durumu yok. Bu makine hiçbir şeyi kabul etmez:

$$L(M) = \emptyset$$

Her durumu kabul durumu olan makine. Aynı makineyi alalım, ama q_0 kabul durumu olsun. Bu öncekinin tersidir. Ret durumu yoktur, her şey kabul edilir:

$$L(M) = \Sigma^*$$

Bütün dizgiler evrensel kümenin kendisi olduğu için dili kısaca evrensel küme diye yazabiliriz.

Yalnızca boş dizgiyi kabul eden makine. Bu kez tersten gidelim: Dil yalnızca boş dizgi olsun, makine ne olmalı? Hiç giriş verilmese bile kabul etmesi gerektiği için başlangıç durumu kabul durumudur. Bu duruma `a` ve `b` ile kendine dönen döngüler koyarsak yine her şeyi kabul eder, bu istediğimiz değildir. Çözüm, her sembolü bir ölü duruma göndermektir. Ölü durumda da her sembolde kalınır.

δ	a	b
q_0 (başlangıç, kabul)	q_1	q_1
q_1 (ölü)	q_1	q_1

$$L(M) = \{\varepsilon\}$$

> **Ders sonrası not:** Hoca bu dili soru olarak yazarken "null set" dedi. Doğrusu boş küme değil, yalnız boş dizgiyi içeren $\{\varepsilon\}$ kümesidir. $\emptyset$'in hiç elemanı yoktur, $\{\varepsilon\}$'un bir elemanı vardır. Bu makineler 10 sembole kadar bütün dizgilerde Python ile tarandı, beklenen dilleri kabul ediyorlar.

12. Tasarım Örneği: `ab` Önekli Dizgiler

Alfabe $\{a, b\}$. İstenen dil, `ab` öneki (prefix) ile başlayan bütün dizgilerdir. Önce üyelerden birkaçını yazalım: `ab`, `abb`, `aba`. `ab` ile başladığı sürece kabul edilir.

Tasarımı adım adım kuralım:

- `a`, sonra `b` gelince bir duruma varmalıyız ve bu kabul durumu olmalıdır. Bu açıktır.
- Bu durumdan sonra başka semboller gelse de kabul durumunda kalırız. Oraya `a` ve `b` için döngü koyarız.
- Şimdi eksik olanlara bakalım: Başlangıçta `b` gelirse ne olur? `a`'dan sonra `a` gelirse ne olur?

Hocanın vurgusu: DFA'da bütün geçişleri yazmalıyız, hiçbirini atlayamayız.

`b` ile başlayan dizgi kabul edilmez. `aa` ile başlayan dizgi de edilmez. İkisi de ölü duruma gitmelidir. Önce iki ayrı ölü durum yazılabilir, ama bunları tek durumda birleştirebiliriz. `b` ile başlayan dizginin devamına bakmaya gerek yoktur, doğrudan reddederiz. Bu yüzden ölü duruma da döngü koyarız.

δ	a	b
q_0 (başlangıç)	q_1	q_3
q_1	q_3	q_2
q_2 (kabul)	q_2	q_2
q_3 (ölü)	q_3	q_3

Tablo 10 sembole kadar bütün dizgilerde Python ile denendi, makine tam olarak `ab` önekli dizgileri kabul ediyor.

13. Tasarım Örneği: `001` İçeren Dizgiler

$L(M)$ `001` altdizgisini (substring) içeren bütün ikili dizgiler olsun. Alfabe $\{0, 1\}$'dir.

13.1 Kümeyi Anlamak

Kümede `001` vardır. Ama `0001` de vardır, `0010` gibi dizgiler de vardır, `11100111` gibi dizgiler de. İstediğimiz, bu parçanın dizginin içinde bir yerde bulunması ve diğer şeylere de izin verilmesidir. Dizgi 1 ile de 0 ile de başlayabilir, ama bir noktada `001` bulunmalıdır.

13.2 Adım Adım Tasarım

Önce en açık kısımdan başlayalım. Başlangıç durumundan 0, sonra 0, sonra 1 okununca bir kabul durumuna varmalıyız. Durumlara q_0, q_1, q_2, q_3 diyelim. `001` bir kez görüldükten sonra ne geldiği önemli değildir. Bu yüzden q_3'e 0 ve 1 için döngü koyarız. Kabul durumuna bir kez gelen hep kabul durumunda kalır.

Biraz kafa karıştıran kısım, baştaki 0 ve 1'lerle ne yapacağımızdır.

- q_0**'da 1 gelirse:** Dizgi 1 ile başlıyor diye reddetmemeliyiz. Hiç 0 görmediğimiz sürece `001` görmüş olamayız. Bu kısmı önemsemiyoruz, q_0'a 1 için döngü koyarız.
- q_1**'de 1 gelirse:** `001` deseninin dışına çıkmışız demektir. Başa dönmeliyiz. Bu yüzden 1 ile q_0'a gidilir.
- q_2**'de 1 gelirse:** `00` görülmüştü, eksik olan 1 de geldi. q_3'e gideriz.
- q_2**'de 0 gelirse:** Durum artık `000` gibidir. q_0'a gidemeyiz, çünkü elimizde 0'lar var. q_0 hiç 0 görmediğimizin durumudur, q_1 bir 0'ın, q_2 iki 0'ın durumudur. Bir 0 daha gelse de sonda yine iki 0 vardır. Bu yüzden q_2'ye 0 için döngü koyarız. Ardından 1 gelirse kabul durumuna gidilir.

δ	0	1	Anlamı
q_0 (başlangıç)	q_1	q_0	Sonda 0 yok
q_1	q_2	q_0	Sonda bir 0 var
q_2	q_2	q_3	Sonda en az iki 0 var
q_3 (kabul)	q_3	q_3	`001` görüldü

Hocanın vurgusu: İşiniz şudur: Bütün durumları tarayın ve her durumda her sembol için ne olacağını yazın.

14. Tasarım Örneği: `001` İçermeyen Dizgiler

Şimdi öncekinin tersine bakalım: `001` altdizgisini içermeyen bütün ikili dizgiler. Yine `001`'i görüp görmediğimizi bilmemiz gerekir, çünkü görenleri reddedeceğiz.

14.1 İlk Deneme ve Düzeltme

Hoca q_0'dan başladı. Yalnız 1'ler geldiği sürece kabul ederiz, bu yüzden q_0 kabul durumudur ve 1 için kendine döner. 0 gelince q_1'e gideriz. `10` gibi dizgiler hâlâ uygundur, yani q_1 de kabul durumudur. Bir 0 daha gelince q_2'ye gideriz. `00` hâlâ uygundur, q_2 de kabul durumudur.

q_1'de 1 gelirse ne olur? Hoca önce q_1'e 1 için döngü koydu. Ama sonra q_2'ye bakarken bir sorun fark etti: Araya 1'ler giren 0'lar, yan yana iki 0 gibi sayılır. Oysa 1 geldiğinde önceki 0'ın artık bir önemi kalmaz. Çözüm, q_1'de 1 gelince başa, q_0'a dönmektir. Bu, bir önceki makinede kullandığımız fikrin aynısıdır.

> **Ders sonrası not:** q_1'de 1 için döngü koymak yanlıştır. Bu yanlış makinede `0101` şu yolu izler: q_0, q_1, q_1, q_2, q_3. Yani dizgi reddedilir. Oysa `0101` içinde `001` yoktur. Doğru makinede yol q_0, q_1, q_0, q_1, q_0 olur ve dizgi kabul edilir.

Geriye q_2 kalır. q_2'de 0 gelirse sorun yoktur, kalırız. 1 gelirse `001` oluşmuştur. Bunu kabul edemeyiz. q_3 durumuna gideriz. q_3 kabul durumu değildir ve ondan sonra ne gelirse gelsin bu ret durumunda kalırız.

δ	0	1
q_0 (başlangıç, kabul)	q_1	q_0
q_1 (kabul)	q_2	q_0
q_2 (kabul)	q_2	q_3
q_3 (ret, tuzak)	q_3	q_3

Hoca bu tür bir tasarımın biraz deneme yanılma istediğini söyledi: Şunu bunu dener, her durum için farklı semboller geldiğinde ne olacağına bakarsınız.

> **Ek bilgi (derste anlatılmadı):** Sonuçtaki makine, Bölüm 13'teki makinenin aynı geçişlerle kabul ve ret durumları yer değiştirmiş hâlidir. Bu genel bir sonuçtur. M bir DFA ise F yerine $Q \setminus F$ almak, $\overline{L(M)}$ dilini kabul eden bir DFA verir. Bunun nedeni, Bölüm 10.2'deki tanımda $\delta^*(q_0, w) \notin F$ koşulunun tam da bunu söylemesidir.

14.2 Makineyi Test Etmek

> **Hocanın vurgusu:** Böyle bir makine yazdıktan sonra farklı testler yazın ve çalışıp çalışmadığına bakın.

Hocanın önerdiği test `00011001` dizgisidir. Bu dizgi `001` içerdiği için reddedilmelidir. Yol şöyledir (ara adım eklendi): $q_0, q_1, q_2, q_2, q_3, q_3, q_3, q_3, q_3$. Dizgi q_3'te biter ve reddedilir. Bölüm 13 ve 14'teki makineler 10 sembole kadar bütün dizgilerde Python ile de denendi, ikisi de doğru dili kabul ediyor.

15. Evde Çalışma Örneği: awa Dili

Hoca bu örneği evde çözmek üzere verdi ve çözümü doğrudan yazdı. Önce çözüme bakmadan kendiniz deneyin.

$$L = \{awa \mid w \in \{a, b\}^*\}$$

Dil, a ile başlayıp a ile biten dizgilerdir.

> **Ders sonrası not:** Hoca dili "a ile başlayıp a ile biten her dizgi" diye anlattı. Tam ifade şudur: En az iki sembollü olmalıdır. Tek sembollü a hem a ile başlar hem a ile biter, ama awa biçiminde değildir. Aşağıdaki makine onu reddeder. Tablo kısa nottan alındı, çünkü diyagram altyazıda okunmuyor.

δ	a	b
q_0 (başlangıç)	q_1	q_3
q_1	q_2	q_1

δ	a	b
q_2 (kabul)	q_2	q_1
q_3 (tuzak)	q_3	q_3

Makine 10 sembole kadar bütün dizgilerde Python ile denendi, tam olarak bu dili kabul ediyor.

16. Düzenli Diller ve Düzenli Olmayan Diller

Gördüğümüz $\{ab, abba\}$ ve $\{a^n b\}$ gibi dilleri kabul eden birer DFA vardır. Bir DFA tarafından kabul edilen dillere düzenli diller (regular languages) denir. Hiçbir DFA tarafından kabul edilmeyen dillere de düzenli olmayan diller (irregular languages) denir. Şimdiye kadar bu derste gördüğümüz bütün örnekler düzenli dillerdi.

> **Ders sonrası not:** Hoca $a^n b$ örneğini burada "$n > 0$" koşuluyla söyledi. Bölüm 7.3'teki makine `b` dizgisini de kabul ettiği için doğru koşul $n \geq 0$'dır.

Düzenli olmayan dile hocanın örneği şudur:

$$\{a^n b^n\}$$

Yani önce n tane `a`, sonra aynı sayıda `b`. Bu dili kabul eden bir DFA tasarlayamazsınız. Hoca, isteyen öğrencilerin bunu evde denemesini önerdi.

> **Ders sonrası not:** Bu sonuç hesapla da denendi: 3 duruma kadar bütün DFA'lar içinde, 8 sembole kadar bütün dizgilerde bu dille tam uyuşan makine çıkmadı. Bu tarama yalnız küçük makineler içindir, genel bir kanıt değildir. Hoca kanıtı bu derste vermedi.

Dersin Toparlaması

Bu derste sonlu otomatın ne olduğunu, durum diyagramıyla nasıl gösterildiğini ve bir dizgi üzerinde adım adım nasıl çalıştığını gördük. Makine bütün girişi okumadan karar vermez. Karar, son durumun kabul durumu olup olmamasına bağlıdır. Ardından otomatın biçimsel tanımını yazdık: $M = (Q, \Sigma, \delta, q_0, F)$. Geçiş fonksiyonu makinenin programıdır ve tablo ya da geçişler kümesi olarak yazılabilir. DFA'da her durumda her sembol için tam bir geçiş vardır. Gerekirse tuzak durum eklenir.

İki temel beceriyi çalıştık. Birincisi, verilen makinenin dilini bulmak: $\{\varepsilon, ab, abba\}$, $\{a^n b \mid n \geq 0\}$, $\{1^n \mid n \text{ çift}\}$ ve özel diller $\emptyset$, Σ^*, $\{\varepsilon\}$. İkincisi, verilen dil için DFA tasarlamak: tek sayıda 1, `ab` önek, `001` içeren ve içermeyen dizgiler. Yöntem hep aynıdır: Örnek dizgiler yazın, deseni anlayın, her durumda her sembolü ele alın, sonra test edin. Son olarak δ^*, $L(M)$ ve düzenli dil kavramlarını tanımladık. Durum sayısından uzun dizgileri kabul eden makinede döngü bulunması, ileride önemli olacak. Bugün DFA'lar bitti. Haftaya deterministik olmayan sonlu otomatlara (NFA) geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.